DG INTERNS HUB · CYBERSECURITY INTERNSHIP

Week 6 · Task 1

OWASP Top 10: Study Notes

Four vulnerabilities every web tester must understand

SQL Injection · Cross-Site Scripting (XSS) · Broken Authentication · Security Misconfiguration

Detail	Value
Name	Ayan Ansari
Intern ID	DG/SEPTEMBER/CYBER/014
Assignment	Week 6 – Web Security & Vulnerability Testing
Task	Task 1 – Study OWASP Top 10, short notes
Date	October 2026

What is OWASP?

OWASP (the Open Worldwide Application Security Project) is a non-profit community that publishes free guidance on web security. Its best-known document is the OWASP Top 10, a list of the most serious and most common web application risks, updated every few years from real data. Security teams, auditors and testers use it as a shared checklist. In the current list, SQL Injection and XSS sit under Injection, Broken Authentication is now called Authentication Failures, and Security Misconfiguration is one of the top categories.

These notes cover the four topics from the brief. Everything here is for learning and defence. Any hands-on testing is done only on practice applications I run myself.

1. SQL Injection (SQLi)

When user input is allowed to change the database question itself.

What it is

A web app often asks its database questions on your behalf. For a login page it may ask: is there a user with this name and this password? SQL Injection happens when the app builds that question by gluing your typed text straight into it. If the text contains database commands, the database cannot tell your input apart from the app's own instructions, so it follows them.

SQL is the language databases use. It is not a flaw in SQL or in the database. The flaw is in how the application builds the query.

> **Simple picture**
> A pharmacy assistant reads out whatever is written on a note. If the note says 'one aspirin and also open the safe', and the assistant obeys, the problem is that the note was trusted as an instruction.

How it happens

- The app joins user input into a query as plain text instead of treating it strictly as data.
- Attackers use form fields, URL parameters, cookies and headers, anywhere input reaches a query.
- Types to know: in-band (results appear on the page), blind (the page only changes slightly, so the attacker infers answers) and out-of-band (data leaves by another channel).

Why it matters

- Attackers can read data they should never see, such as users, emails and password hashes.
- They may change or delete records, or log in without a valid password.
- In bad cases the database server itself can be taken over.
- SQLi has led to some of the largest data breaches on record.

How to prevent it

- Use parameterised queries (prepared statements) so input is always treated as data. This is the main fix.
- Use an ORM or safe database library correctly, and never build queries by string joining.
- Validate input against what is expected (type, length, allowed characters) as a second layer.
- Give the app's database account the least privilege it needs, never an admin account.
- Hide detailed database errors from users and log them privately instead.
- Test regularly with scanners and code review.

OWASP reference: OWASP Top 10 category: Injection (A03:2021).

2. Cross-Site Scripting (XSS)

When a website shows untrusted input back to visitors as live page code.

What it is

XSS means an attacker gets their own script to run inside another person's browser, on a trusted website. The browser cannot tell that the script was not written by the site, so it runs with the site's full trust. The target of the attack is usually the visitor, not the server.

It happens when a page puts user-supplied text into its HTML without making it safe first.

> **Simple picture**
> A noticeboard at a school lets anyone pin notes. If a prankster pins a note that secretly tells every reader to hand over their ID, the board looked official, so people obeyed.

How it happens

- Reflected XSS: the malicious input is part of a link or request and is bounced straight back in the response. The victim usually has to click a crafted link.
- Stored XSS: the input is saved by the site (a comment, profile or message) and runs for every visitor who loads that content. It is the more dangerous kind.
- DOM-based XSS: the page's own JavaScript writes untrusted data into the page in the browser, without the server being involved.

Why it matters

- Session cookies can be stolen, letting an attacker act as the victim.
- Pages can be altered to show fake login forms or false content.
- Actions can be performed in the victim's name, such as posting or changing settings.
- Malware or unwanted redirects can be delivered through a trusted site.

How to prevent it

- Output encoding: convert special characters to safe text before putting data in a page. Do this for the right context (HTML, attribute, JavaScript, URL).
- Use frameworks that escape output by default, and avoid unsafe 'raw HTML' features.
- Add a Content Security Policy (CSP) to limit which scripts the browser may run.
- Mark session cookies HttpOnly and Secure so scripts cannot read them.
- Validate and sanitise input, and sanitise rich text with a well-known library.
- Test with scanners and manual review.

OWASP reference: OWASP Top 10 category: Injection (A03:2021).

3. Broken Authentication

Weak ways of proving who you are, or of staying logged in.

What it is

Authentication is how a system checks that you are who you claim to be: passwords, one-time codes, login sessions. Broken Authentication covers any weakness that lets an attacker pretend to be another user, either by guessing or stealing credentials, or by abusing how sessions are handled.

OWASP now groups this under 'Identification and Authentication Failures'.

> **Simple picture**
> A building gives staff a keycard but never checks it at the door, never turns old cards off and lets anyone try any number of codes. The lock exists, but it does not protect anything.

How it happens

- No limit on login attempts, so passwords can be guessed again and again (brute force) or reused leaked passwords can be tried in bulk (credential stuffing).
- Weak or default passwords are accepted, such as simple words or admin/admin.
- Session IDs are predictable, shown in the URL, or never expire, so a stolen ID works for a long time.
- Passwords are stored in plain text or with weak hashing.
- Password reset and 'forgot password' flows that reveal too much or can be abused.
- Logging out does not really end the session.

Why it matters

- Attackers take over user or admin accounts and see private data.
- One compromised account can be used to reach many others, because people reuse passwords.
- The business may face legal and trust problems after a takeover.

How to prevent it

- Require strong passwords and check them against lists of known leaked passwords.
- Turn on multi-factor authentication (MFA), especially for admin accounts.
- Limit login attempts and add delays or lockouts, with alerting.
- Create long random session IDs, send them only over HTTPS, set cookies Secure and HttpOnly, and expire them.
- Regenerate the session ID after login and destroy it fully at logout.
- Store passwords only as salted hashes made with a slow algorithm such as bcrypt or Argon2.
- Use the same generic message for 'wrong user' and 'wrong password'.

OWASP reference: OWASP Top 10 category: Identification and Authentication Failures (A07:2021).

4. Security Misconfiguration

Safe software set up in an unsafe way.

What it is

Many breaches need no clever trick: something was simply left open or left at its default. Security Misconfiguration means the application, web server, database or cloud service is set up in a way that weakens security. It is one of the most common findings in real assessments.

In Week 5 my own lab server showed small examples: the server version was visible, TRACE was switched on, and two browser protection headers were missing.

> **Simple picture**
> A shop installs an excellent lock, then leaves the factory-default code on it, a spare key under the mat and the back window open because nobody checked.

How it happens

- Default accounts and passwords are left unchanged.
- Unnecessary features, sample pages, admin consoles or open ports are left switched on.
- Detailed error messages show stack traces, file paths or software versions to visitors.
- Security headers (such as X-Frame-Options, X-Content-Type-Options, CSP, HSTS) are missing.
- Directory listing is enabled, so anyone can browse a folder's files.
- Cloud storage is public by mistake, or software is left out of date.

Why it matters

- Information leaks help attackers plan their next move.
- Open admin panels or default logins can hand over full control.
- Public storage can expose large amounts of data without any 'hacking' at all.

How to prevent it

- Build a hardened baseline and use the same secure setup everywhere (servers, containers, cloud).
- Remove or disable features, services, samples and accounts that are not needed.
- Change all default credentials and close unused ports with a firewall.
- Show generic error pages to users and keep details in private logs.
- Add security headers and turn on HTTPS.
- Patch and update regularly, and review settings often, ideally with automated checks.
- Re-scan after changes. A fix is only done when a re-test proves it.

OWASP reference: OWASP Top 10 category: Security Misconfiguration (A05:2021).

Quick comparison

Vulnerability	Root cause	Main fix
SQL Injection	Input is joined into a database query as text	Parameterised queries
XSS	Untrusted input is shown as live page code	Output encoding plus CSP
Broken Authentication	Weak login and session handling	MFA, strong sessions, rate limits
Security Misconfiguration	Insecure defaults or settings left in place	Hardened baseline and regular review

What I take from this

- Three of the four are not clever tricks. They come from trusting input, trusting defaults or skipping a basic check.
- The fixes are mostly well known and cheap when done during development, and expensive after a breach.
- Learning to attack in order to defend only makes sense on systems I own or have permission to test.

Sources

OWASP Top 10:2021 (owasp.org/Top10) and the OWASP Cheat Sheet Series (cheatsheetseries.owasp.org). Notes are written in my own words for learning.